TCP: Transport Layer

Reliable Data Transfer

A reliable channel means that no transferred data bits are corrupted (flipped from 0 to 1, or vice versa) or lost, and all are delivered in the order in which they were sent. This is precisely the service model offered by TCP to the Internet applications that invoke it.

It is the responsibility of a **reliable data transfer protocol** to implement this service abstraction. This task is made difficult by the fact that the layer *below* the reliable data transfer protocol may be unreliable. For example, TCP is a reliable data transfer protocol that is implemented on top of an unreliable (IP) end-to-end network layer. More generally, the layer beneath the two reliably communicating end points might consist of a single physical link (as in the case of a link-level data transfer protocol) or a global internetwork (as in the case of a transport-level protocol). For our purposes, however, we can view this lower layer simply as an unreliable point- to-point channel.

Q1. What does reliability mean for data transferred over the internet?

Before developing a protocol for reliably communicating over such a channel, first consider how people might deal with such a situation. Consider how you yourself might dictate a long message over the phone. In a typical scenario, the message taker might say “OK” after each sentence has been heard, understood, and recorded. If the message taker hears a garbled sentence, you’re asked to repeat the garbled sentence. This message-dictation protocol uses both **positive acknowledgments** (“OK”) and **negative acknowledgments** (“Please repeat that.”). These control messages allow the receiver to let the sender know what has been received correctly, and what has been received in error and thus requires repeating.

Similarly, during network communication a protocol must exist to give feedback about whether the receiver got the message without error or with error or none at all.

1. Error Detection: Some extra bits are sent along in the data packet to the receiver so that the receiving end can check whether it received the data without any corruption (bit flips). We discussed parity bits in class. There are also checksum bits that are sent in the packet header at the transport layer.

Q2. What is a checksum and how does it help us to detect any possible corrupt bits in the data that has been transferred? You may use the internet to help you explain it briefly.

2. Receiver feedback. Since the sender and receiver are typically executing on different end systems, possibly separated by thousands of miles, the only way for the sender to learn of the receiver's view of the world (in this case, whether or not a packet was received correctly) is for the receiver to provide explicit feedback to the sender. The positive (ACK) and negative (NAK) are sent to the receiver for this purpose. If an ACK packet is received the sender knows that the most recently transmitted packet has been received correctly and thus the protocol returns to the state of waiting for data from the upper layer. If a NAK is received, the protocol retransmits the last packet and waits for an ACK or NAK to be returned by the receiver in response to the retransmitted data packet. It is important to note that when the sender is in the wait-for-ACK-or-NAK state, it cannot get more data from the upper layer.

Q3. How long should the ACK or NAK message be? How many fields or bytes should it have to convey the acknowledgement back to the sender? (Think and give your own opinion)

Q4. What is meant by an "upper layer" here? Name the layer being talked about?

Q5. Can the ACK and NAK packets also be corrupted like data packets?

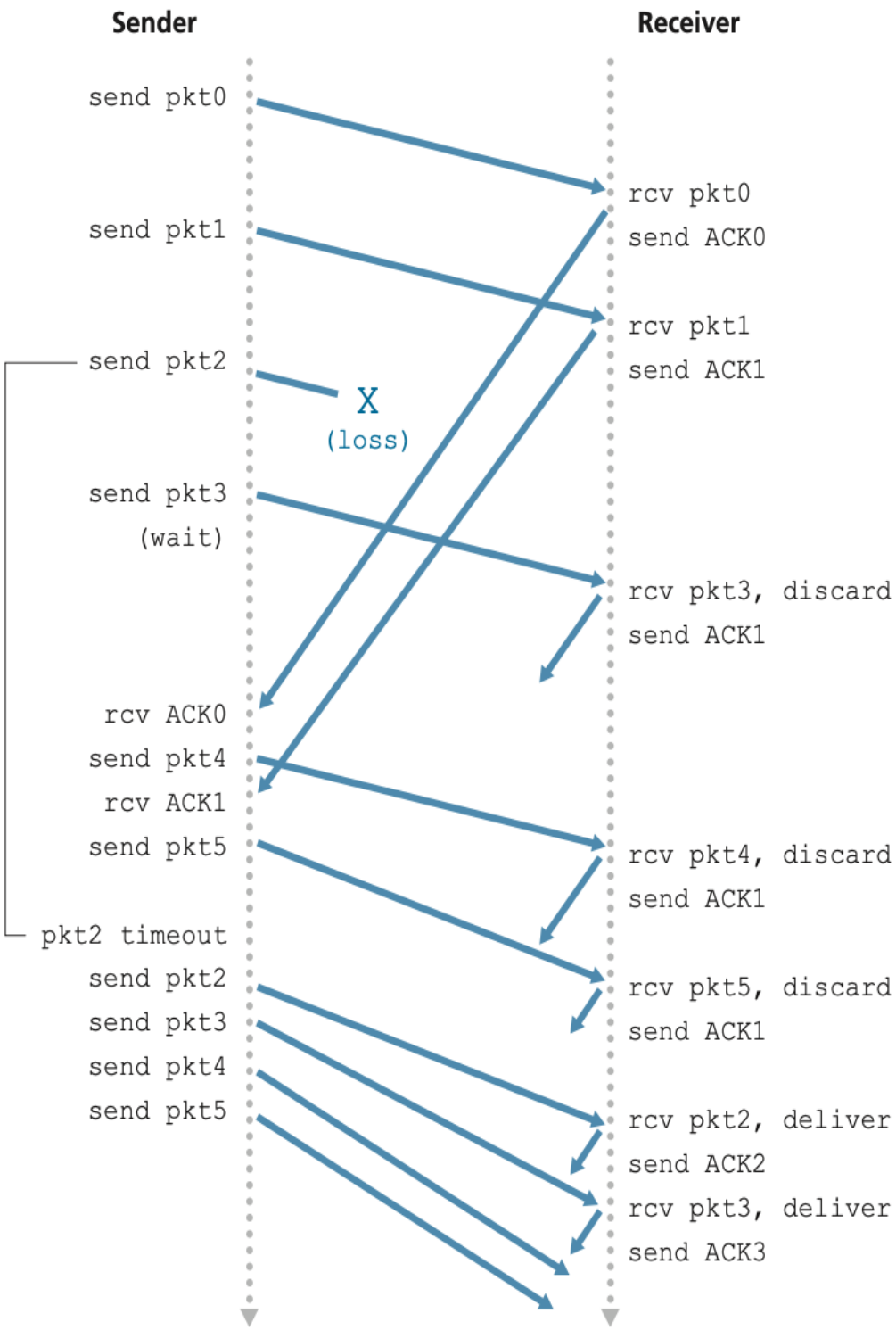
If no ACK is received by the sender, it does not know whether a data packet was lost, an ACK was lost, or if the packet or ACK was simply overly delayed. The remedy for any of these at the sender's end is to retransmit the packet. How will the receiver know whether this is a retransmitted packet or a following packet in sequence? For this TCP adds a sequence number to its header, this sequence number, numbers its packets and helps the receiver to know whether a packet is being transmitted for a first time or whether it is a retransmission. The unreliable IP network may also deliver packets in a different order at the receiver than the other in which they were sent out by the sender. Sequence number will also help TCP to order and communicate packets to the application layer in the order in which they were sent by the sender.

Q6. Can you think of a reason why packets sent from the sender to the receiver end host will be out of order?

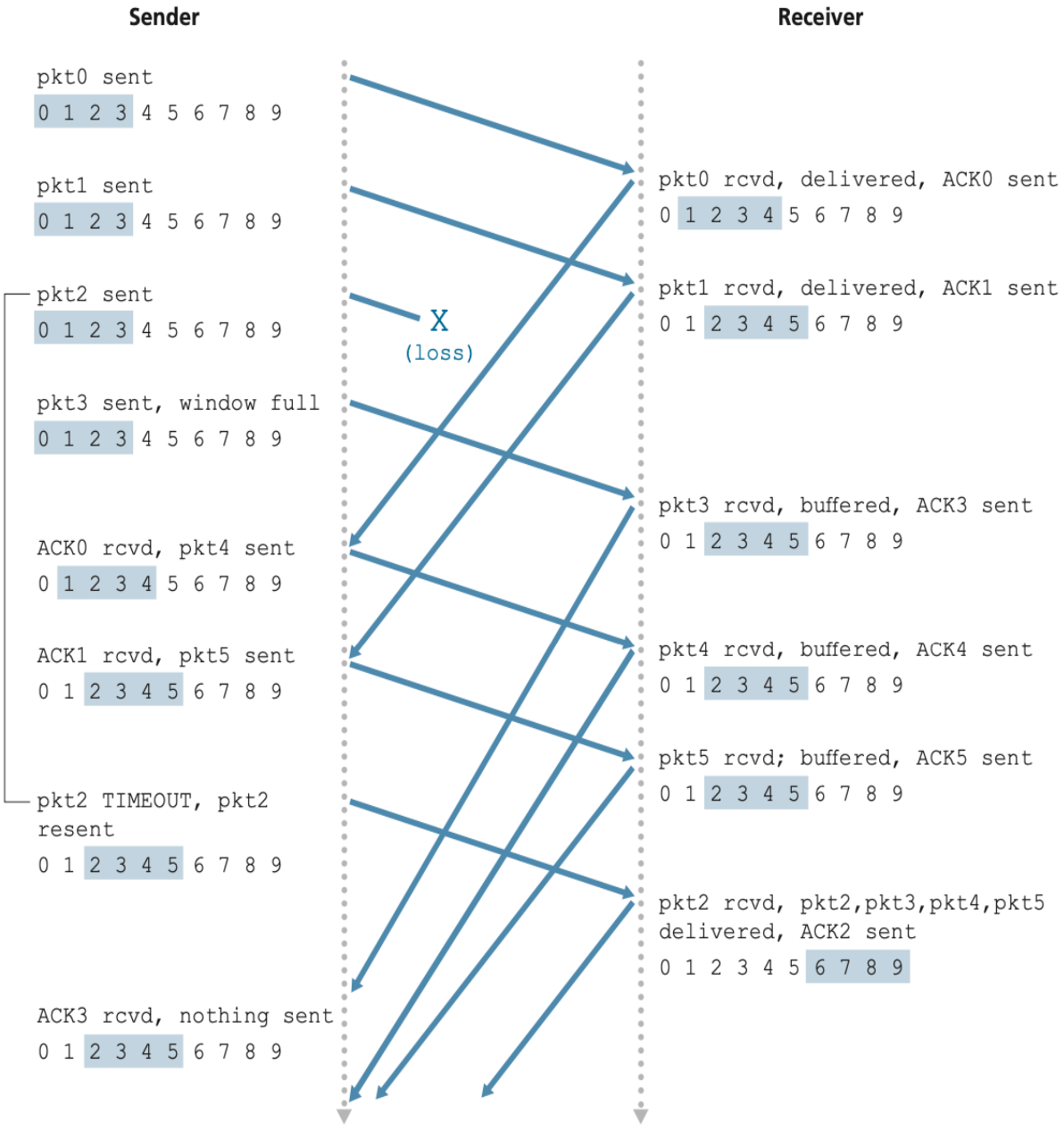
If we implement send and wait-for- ACK strategy for TCP then its sending rate is severely limited. A lot of the server time will only be spent waiting.

Q7. a) What will happen if we allow TCP to send upto a set number of packets without waiting for acknowledgement for them? (This is called PIPELINING)
b) What can go wrong if this number is very large?
c) What will be the effect of this number being very small?
d) Why not an unlimited number of packets be allowed to flood the link from the sender?

Problem with windowing:

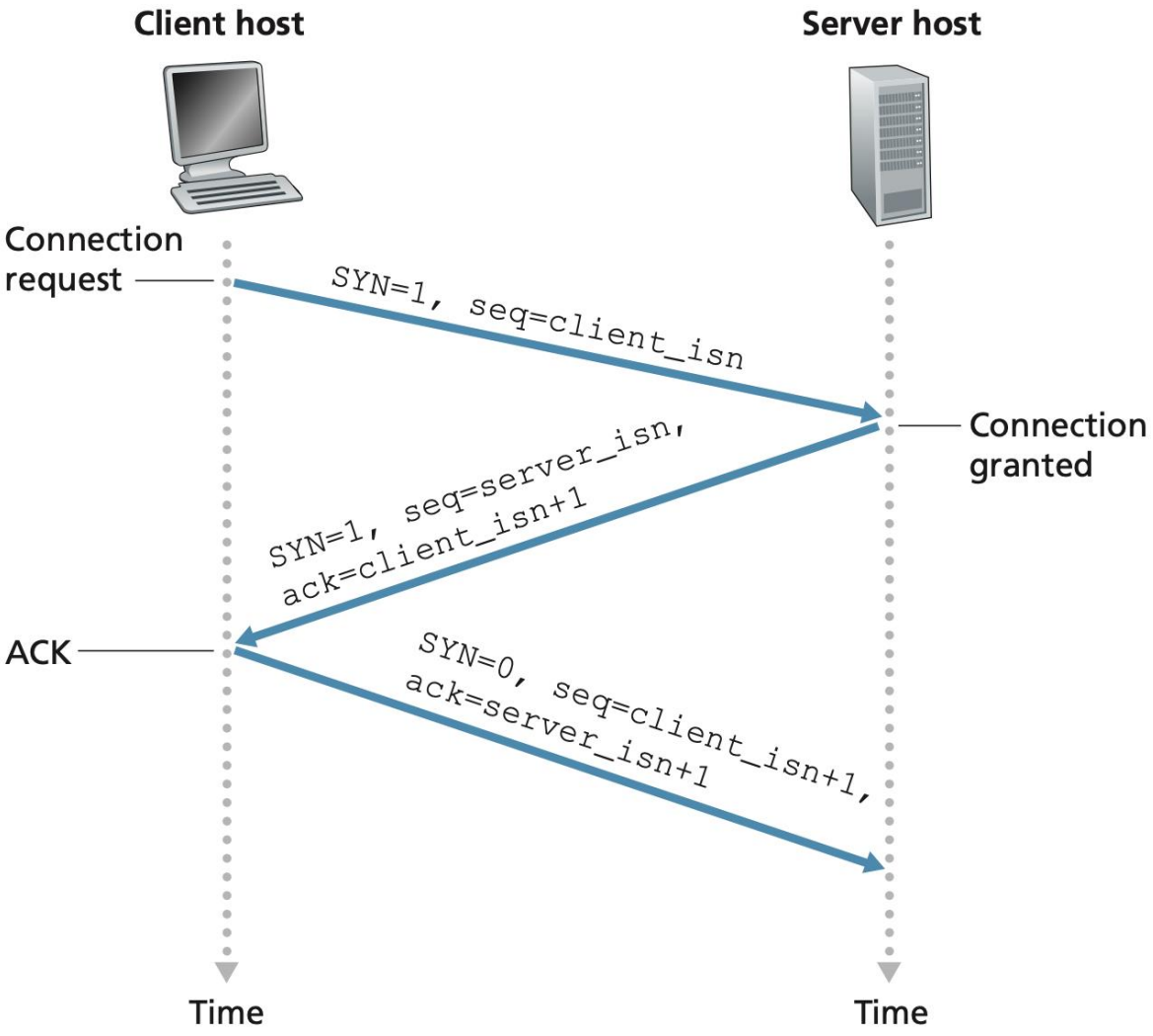


Problem solved with **buffering at receiver** and **selective resend from sender:**



TCP Connection:

The client application process first informs the client TCP that it wants to establish a connection to a process in the server. The TCP in the client then proceeds to establish a TCP connection with the TCP in the server in the following manner:



- Step 1. The client-side TCP first sends a special TCP segment to the server-side TCP. This special segment contains no application-layer data. But one of the flag bits in the segment's header (see Figure 3.29), the SYN bit, is set to 1. For this reason, this special segment is referred to as a SYN segment. In addition, the client randomly chooses an initial sequence number (client_isn) and puts this number in the sequence number field of the initial TCP SYN segment. This seg- ment is encapsulated within an IP datagram and sent to the server. There has been

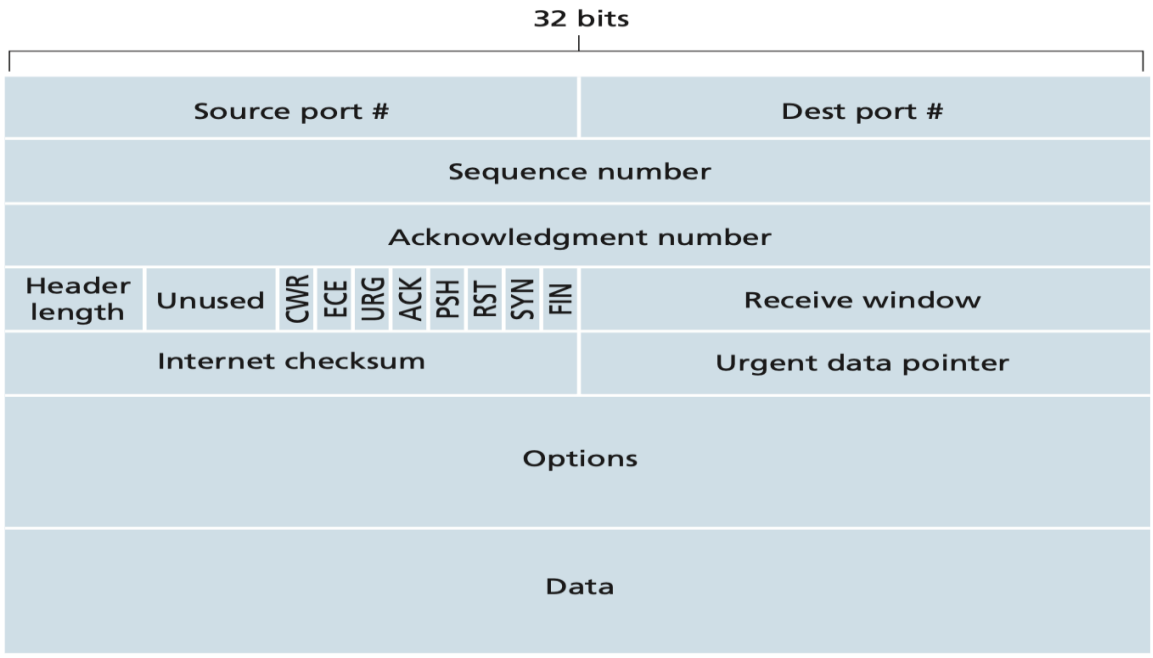
considerable interest in properly randomizing the choice of the client_isn in order to avoid certain security attacks [CERT 2001–09; RFC 4987].

- Step 2. Once the IP datagram containing the TCP SYN segment arrives at the server host (assuming it does arrive!), the server extracts the TCP SYN segment from the datagram, allocates the TCP buffers and variables to the connection, and sends a connection-granted segment to the client TCP. (We'll see in Chapter 8 that the allocation of these buffers and variables before completing the third step of the three-way handshake makes TCP vulnerable to a denial-of-service attack known as SYN flooding.) This connection-granted segment also contains no application-layer data. However, it does contain three important pieces of information in the segment header. First, the SYN bit is set to 1. Second, the acknowledgment field of the TCP segment header is set to client_isn+1. Finally, the server chooses its own initial sequence number (server_isn) and puts this value in the sequence number field of the TCP segment header. This connection-granted segment is saying, in effect, "I received your SYN packet to start a connection with your initial sequence number, client_isn. I agree to establish this connection. My own initial sequence number is server_isn." The connection-granted segment is referred to as a SYNACK segment.
- Step 3. Upon receiving the SYNACK segment, the client also allocates buffers and variables to the connection. The client host then sends the server yet another segment; this last segment acknowledges the server's connection-granted segment (the client does so by putting the value server_isn+1 in the acknowledgment field of the TCP segment header). The SYN bit is set to zero, since the connection is established. This third stage of the three-way handshake may carry client-to- server data in the segment payload.

Once these three steps have been completed, the client and server hosts can send segments containing data to each other. In each of these future segments, the SYN bit will be set to zero. Note that in order to establish the connection, three packets are sent between the two hosts. For this reason, this connection-establishment procedure is often referred to as a **three-way handshake**.

Q 8. Why do you think the handshake is three ways and not two ways? Why not exchange just two messages for handshake?

TCP Segment Structure



Sequence number and acknowledgement number fields:

TCP views data as an unstructured, but ordered, stream of bytes. TCP's use of sequence numbers reflects this view in that sequence numbers are over the stream of transmitted bytes and *not* over the series of transmitted segments. The sequence number for a segment is therefore the byte-stream number of the first byte in the segment. Let's look at an example. Suppose that a process in Host A wants to send a stream of data to a process in Host B over a TCP connection. The TCP in Host A will implicitly number each byte in the data stream. Suppose that the data stream consists of a file consisting of 500,000 bytes, that the MSS is 1,000 bytes, and that the first byte of the data stream is numbered 0. As shown in Figure 3.30, TCP constructs 500 segments out of the data stream. The first segment gets assigned sequence number 0, the second segment gets assigned sequence number 1,000, the third segment gets assigned sequence number 2,000, and so on. Each sequence number is inserted in the sequence number field in the header of the appropriate TCP segment.

Now let's consider acknowledgement numbers. These are a little trickier than sequence numbers. Recall that TCP is full-duplex, so that Host A may be receiving data from Host B while it sends data to Host B (as part of the same TCP connection). Each of the segments that arrive from Host B has a sequence number for the data flowing from B to A. *The acknowledgment number that Host A puts in its segment is the sequence number of the next byte Host A is expecting from Host B.* It is good to look at a few examples to understand what is going on here. Suppose that Host A has received all bytes numbered 0 through 535 from B and suppose that it is about to send a segment to Host B. Host A is waiting for byte 536 and all the subsequent bytes in Host B's data stream. So Host A puts 536 in the acknowledgment number field of the segment it sends to B.

As another example, suppose that Host A has received one segment from Host B containing bytes 0 through 535 and another segment containing bytes 900 through 1,000. For some reason Host A has not yet received bytes 536 through 899. In this example, Host A is still waiting for byte 536 (and beyond) in order to re-create B's data stream. Thus, A's next segment to B will contain 536 in the acknowledgment number field. Because TCP only acknowledges bytes up to the first missing byte in the stream, TCP is said to provide cumulative acknowledgments.

Q9. What should TCP do with the segment containing bytes 900 through 1,000 that it has received out of order?